Cloud-Native AI Model Deployment Platform: A Microservice-Based Architecture for Serving, Scaling and Monitoring Machine Learning Models

Vemuri Venkata Naga Sai Sri Harshad[1], Pinnamaneni Rohith Venkata Sai[1], Ventrapragada Jyothi Swaroop[1], Mullapudi Gowtham Chowdary[1], R. Ranjith Kumar[1]

[1]Department of Computer Science and Engineering (CSE-3), Koneru Lakshmaiah Education Foundation (Deemed to be University), Green Fields, Vaddeswaram, Guntur, Andhra Pradesh 522302, India.
Corresponding author: Pinnamaneni Rohith Venkata Sai (2300031556)

Abstract. Deploying machine learning (ML) models in production is difficult when it relies on monolithic services and manual release steps: such systems are hard to scale, version, secure and monitor. This paper presents the design and prototype implementation of a Cloud-Native AI Model Deployment Platform that manages the lifecycle of ML models through containerised microservices. The platform comprises a Spring Boot API gateway that enforces JWT authentication and role-based access control, a FastAPI model registry backed by PostgreSQL, a FastAPI inference service, a React.js management dashboard and a Raspberry Pi 4 edge client. Services are packaged as Docker images, described by Kubernetes manifests with horizontal pod autoscaling, built and released through GitHub Actions workflows, and observed with Prometheus and Grafana. The design is grounded in a review of sixteen recent studies on MLOps, Kubernetes-based inference serving, API-gateway security and edge computing. The prototype was verified with 23 automated tests across the registry and inference services, all of which pass, and with an interactive dashboard that exercises model registration, deployment, scaling, inference and monitoring workflows. The inference engine, dashboard metrics and edge client are currently simulated, and load testing on a live cluster remains future work. The results show that a small team can assemble a coherent cloud-native serving platform from standard open-source components, and they identify the integration steps needed to turn the prototype into a production-ready system.

Keywords: Cloud-native computing; MLOps; Kubernetes; microservices; model serving

1. Introduction

Machine learning (ML) now supports decision making in healthcare, finance, agriculture, manufacturing and smart cities, so organisations increasingly need to move trained models into production reliably. Kreuzberger et al. (2023) describe the discipline that addresses this problem, machine learning operations (MLOps), as the combination of development, deployment and operations practices needed to automate and operationalise ML products. In practice, however, many teams still serve models from a single monolithic application that is released by hand. Shivashankar et al. (2025) and Eken et al. (2024) report that such setups struggle with scalability, maintainability, model versioning, monitoring and governance, and Shivashankar et al. identify container-based microservice isolation as the main remedy for maintainability problems.

Cloud-native technologies offer a way to address these limitations. Containers make a service portable, orchestrators such as Kubernetes provide self-healing, load balancing and horizontal scaling, and automated pipelines turn a code change into a tested, deployed release. The literature supports this direction from several angles. Garg et al. (2022) present CI/CD pipelines for automated ML model deployment, and Pandey et al. (2022) deploy ML models on Kubernetes with Kubeflow across several cloud providers. Marcos-Mercadé et al. (2026) benchmark modern MLOps frameworks, while Xu et al. (2025) and Fehlis et al. (2026) show

cloud-native and microservice architectures for LLM and document-AI serving. Ali (2026) compares a FastAPI service with NVIDIA Triton on Kubernetes, and Jin and Yang (2025) and Rubak and Taheri (2023) study scaling strategies for inference services and microservices. Punniyamoorthy et al. (2025) discuss secure, governed API gateways, Naveed et al. (2025) survey how practitioners monitor ML systems, Abdelfattah et al. (2026) address load testing of model serving, and Khouas et al. (2024) survey ML at the edge.

Most of this work studies one concern in isolation, such as pipelines, inference performance, gateways or monitoring. Students and small teams who want to understand or reproduce a complete platform therefore have to integrate the pieces themselves. The aim of this work is to design and prototype a single, small but end-to-end platform that combines a model registry, a secure gateway, an inference service, container orchestration, CI/CD, monitoring and an edge client, using only open-source components and free-tier cloud resources.

The contributions of this paper are: (i) a microservice architecture for deploying and managing ML models, derived from the reviewed literature; (ii) a working prototype consisting of three backend services, a React.js dashboard, Kubernetes manifests with autoscaling and three GitHub Actions workflows; (iii) a verification of the prototype with automated tests and an interactive dashboard; and (iv) an honest account of what is simulated and what remains to be validated on a live cluster. The rest of the paper is organised as follows. Section 2 describes the design and methods, Section 3 presents and discusses the results, and Section 4 concludes.

2. System Design and Methods

2.1 Architecture overview

The platform follows a microservice architecture in which every capability is an independent, containerised service (Figure 1). Users and the Raspberry Pi 4 edge client never talk to the backend services directly. All traffic enters through the API gateway, which authenticates the caller and forwards the request to the model registry or the inference service. The services run inside a Kubernetes cluster behind an ingress, and a Prometheus and Grafana stack observes them. Table 1 lists the components and their responsibilities.

Table 1 Platform components, technologies and responsibilities

Component	Technology	Responsibility
API gateway	Spring Boot, Spring Security	JWT login, role checks, request routing
Model registry	FastAPI, SQLAlchemy, PostgreSQL	Model metadata, versions, upload
Inference service	FastAPI, Prometheus instrumentator	Prediction endpoints, health, metrics
Dashboard	React.js, Vite, Tailwind, Recharts	Registry, deployments, inference, monitoring
Edge client	Raspberry Pi 4 (REST client)	Sends data to cloud inference APIs
Orchestration	Docker, Kubernetes, HPA, ingress	Packaging, scheduling, autoscaling
CI/CD	GitHub Actions, GHCR	Test, scan, build, push, deploy
Observability	Prometheus, Grafana	Metrics collection and dashboards

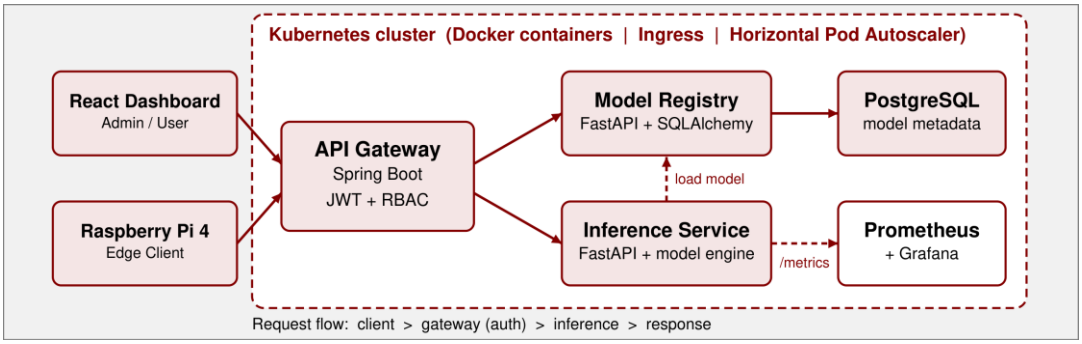


Figure 1 System architecture of the Cloud-Native AI Model Deployment Platform

2.2 Backend microservices

The API gateway is a Spring Boot application. A login endpoint issues a signed JWT that carries the user name and role and expires after eight hours. A security filter validates the token on every request and applies role-based access control (RBAC): any authenticated user may list models and request predictions, whereas deleting, patching and uploading models is restricted to the ADMIN role. A proxy controller then forwards model requests to the registry and inference requests to the inference service.

The model registry is a FastAPI service that stores model name, version, framework, size and status through SQLAlchemy. It exposes create, read, update, delete and upload endpoints under /api/v1/models and uses PostgreSQL in the Docker Compose and Kubernetes deployments. The inference service is a second FastAPI service that lists the available models and serves predictions under /api/v1/inference/{model_name}. It also exposes a health endpoint and Prometheus metrics. In the current prototype its model engine is a mock that returns representative outputs for four sample models (an image classifier, a sentiment analyser, a fraud detector and an object detector). It is the component to be replaced with real model loading, and its interface was designed so that this replacement does not affect the gateway or the clients.

2.3 Containerisation and orchestration

Each service has its own Dockerfile, and Docker Compose starts the complete stack locally, including PostgreSQL, Prometheus and Grafana. For cluster deployment the project provides 24 Kubernetes manifests: a namespace, secrets, deployments and services for every component, persistent volume claims for the database and the registry, an ingress, and monitoring deployments. The deployments define liveness and readiness probes and CPU and memory requests and limits. A horizontal pod autoscaler (HPA) is attached to the gateway, the registry and the inference service (Table 2). The inference service has the highest minimum replica count and the lowest CPU threshold, because it is the most compute-intensive and the most latency-sensitive component. This follows the guidance of Jin and Yang (2025) on combining load balancing with automated scaling for inference services.

Table 2 Horizontal pod autoscaler settings defined in the Kubernetes manifests

Service	Replicas (min-max)	Scale-out trigger
API gateway	2-8	CPU 70% or memory 80%
Model registry	2-6	CPU 70% or memory 80%
Inference service	3-10	CPU 65% or memory 75%

2.4 CI/CD pipeline

Three GitHub Actions workflows automate delivery (Figure 2). The CI workflow runs on every push and pull request and executes the Python test suites, builds the Java gateway and builds the frontend. The security workflow builds each image and scans it with Trivy, audits the Python dependencies with pip-audit and the frontend dependencies with npm audit. The CD workflow builds and pushes one image per service to the GitHub Container Registry, applies the Kubernetes manifests with image tags set to the commit hash, waits for the rollouts to finish and runs a smoke test. This structure follows the reference architectures for automated ML deployment described by Garg et al. (2022) and Kreuzberger et al. (2023).

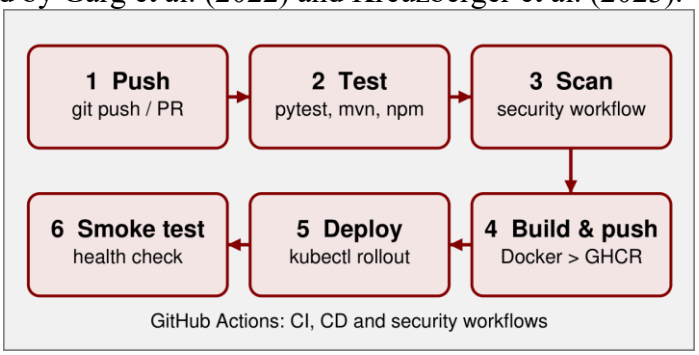


Figure 2 CI/CD pipeline implemented with GitHub Actions

2.5 Monitoring, dashboard and edge client

Prometheus scrapes the gateway, the registry and the inference service every 15 seconds, and Grafana is provisioned with Prometheus as its data source. Naveed et al. (2025) report that practitioners monitor latency, resource use and data quality with exactly this kind of toolchain, which motivated the choice. The React.js dashboard has seven pages: dashboard overview, model registry, deployments, inference testing, monitoring, API gateway and edge client. It allows a user to sign in, register and delete models, deploy and undeploy them, scale, stop and restart deployments, submit inference requests, and inspect charts, gateway logs and edge-device status. In this prototype the dashboard runs on simulated data held in a shared client-side store, so that every control can be demonstrated without a running cluster. The Raspberry Pi 4 edge client represents the edge scenario surveyed by Khouas et al. (2024): a resource-constrained device that sends data to the cloud inference service over the gateway.

2.6 Verification method

The backend services were verified with automated pytest suites that call the FastAPI applications through their HTTP interfaces, using a temporary SQLite database for the registry tests. The dashboard was verified by building it for production, linting it, and exercising its main workflows in a browser: uploading and deploying a model, restarting a deployment, running inference, and checking that the resulting entries appear in the gateway log and the notification list.

3. Results and Discussion

3.1 Implementation outcome

The prototype implements the full request path from the dashboard to the registry and inference services through the gateway, together with the deployment, CI/CD and monitoring assets needed to run it. Table 3 summarises the status of each component. Components marked as simulated are discussed in Section 3.4.

Table 3 Implementation status of the platform components

Component	Evidence	Status

API gateway	JWT login, RBAC, proxy routes	Implemented
Model registry	CRUD and upload, 13 tests	Implemented
Inference service	Predict and metrics, 10 tests	Implemented (mock engine)
Dashboard	7 pages, production build passes	Prototype (mock data)
Kubernetes	24 manifests incl. 3 HPAs	Defined, not yet deployed
CI/CD	3 workflows (CI, security, CD)	Defined
Monitoring	Prometheus and Grafana config	Configured
Edge client	Edge page in dashboard	Simulated

3.2 Functional verification

All 23 automated tests pass: 10 for the inference service and 13 for the model registry. In the browser, the main workflows behaved as intended. Registering a model adds it to the registry with status pending; deploying it creates a running deployment, updates the dashboard counters and adds notification and gateway log entries; stopping, restarting and scaling a deployment change its status and replica count; and an inference request is accepted only for a running deployment and increments its request counter. Invalid JSON payloads are rejected with a 400 response. When the dashboard is used with the USER role, a delete request is rejected with a 403 response, which mirrors the ADMIN-only rule enforced by the gateway.

3.3 Scalability, security and observability design

The platform design addresses the three concerns raised in the introduction. For scalability, the three HPAs allow the inference service to run between three and ten replicas, and each service is stateless apart from the database, so replicas can be added without coordination. For security, authentication and authorisation are centralised in the gateway, secrets are kept in Kubernetes secrets, and images and dependencies are scanned on every pipeline run. For observability, all three backend services are scraped by Prometheus and visualised in Grafana. These choices correspond to the architectures recommended by Xu et al. (2025) for cloud-native inference serving, by Punniyamoorthy et al. (2025) for governed gateways, and by Kreuzberger et al. (2023) for MLOps in general. Because no load test has been run, the paper does not claim measured throughput, latency or scaling behaviour.

3.4 Limitations

Four limitations should be kept in mind. First, the inference service uses a mock model engine, so predictions are placeholders rather than outputs of trained models. Second, the dashboard metrics, deployment status and edge-client data are simulated; Prometheus is configured but not yet connected to the dashboard. Third, the Kubernetes manifests and workflows are defined and syntactically complete but have not yet been validated on a live AWS, Azure or GCP cluster. Fourth, no load or latency benchmarking has been performed, so the autoscaling thresholds in Table 2 are design choices rather than tuned values. Also, the gateway currently authenticates against a small built-in user list and its signing key is read from configuration; both must be replaced by a proper identity store and secret management before production use. Abdelfattah et al. (2026) show that realistic load testing is essential for capacity planning of serving systems, which makes this the highest-priority next step.

3.5 Discussion

The prototype shows that the components recommended separately in the literature can be combined into one coherent system with modest effort. A small number of standard building blocks (FastAPI, Spring Boot, PostgreSQL, Docker, Kubernetes, GitHub Actions, Prometheus and Grafana) cover the model lifecycle, and the use of a gateway and a registry as separate services keeps the inference path independent of model management. Compared with a

monolithic model server, the main costs are additional moving parts and the need for container and cluster skills; the main benefits are independent scaling and release of each service, a single place to enforce security, and a clear place for monitoring. The results are limited to the architecture and functional level, and should be extended with measurements, for example by comparing reactive autoscaling with the predictive approach of Rubak and Taheri (2023), and by evaluating a dedicated serving runtime against the FastAPI service as in Ali (2026).

4. Conclusions

We designed and built a prototype cloud-native platform that covers the model lifecycle: registering, containerising, deploying, serving, scaling and monitoring ML models through loosely coupled microservices. The platform combines a JWT and RBAC protected gateway, a model registry, an inference service, a dashboard and an edge client, delivered through Docker, Kubernetes manifests with autoscaling and three GitHub Actions workflows. Twenty-three automated tests pass, and the dashboard demonstrates the main management workflows. The inference engine, metrics and edge client are still simulated and the deployment has not yet been validated on a live cluster.

Future work will (i) load real PyTorch, TensorFlow or ONNX model artifacts from the registry into the inference service; (ii) deploy the stack on a cloud free tier and benchmark throughput, latency and autoscaling under load; (iii) connect live Prometheus metrics to the dashboard and add alerting; (iv) connect the physical Raspberry Pi 4 edge client; and (v) add canary rollouts and model-drift monitoring.

Acknowledgements

The authors thank their supervisor, Mr. R. Ranjith Kumar, and the Department of Computer Science and Engineering, KL University, for their guidance and for the laboratory and cloud resources used in this work.

References

Abdelfattah, A.S., Tirumalai, N., Mohanan, I., Li, X., Wang, P., Dhurjati, D., Sung, E., 2026. Load Testing for Machine Learning Model Serving Systems at Scale. arXiv preprint.

Ali, R., 2026. Scalable and Secure AI Inference in Healthcare: A Comparative Benchmarking of FastAPI and Triton Inference Server on Kubernetes. arXiv:2602.00053.

Eken, B., Pallewatta, S., Tran, N.K., Tosun, A., Babar, M.A., 2024. A Multivocal Review of MLOps Practices, Challenges and Open Issues. arXiv:2406.09737.

Fehlis, Y., Bengfort, B., Si, Z., Eyorokon, V., Roman, P., Deziel, P., et al., 2026. Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production. arXiv:2605.18818.

Garg, S., Pundir, P., Rathee, G., Gupta, P.K., Garg, S., Ahlawat, S., 2022. On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps. arXiv preprint.

Jin, Y., Yang, Z., 2025. Scalability Optimization in Cloud-Based AI Inference Services: Strategies for Real-Time Load Balancing and Automated Scaling. arXiv:2504.15296.

Khouas, A.R., Bouadjenek, M.R., Hacid, H., Aryal, S., 2024. Training Machine Learning Models at the Edge: A Survey. arXiv:2403.02619.

Kreuzberger, D., Kühl, N., Hirschl, S., 2023. Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access.

Marcos-Mercadé, J., Lopez-Novoa, U., Egana Aranguren, M., 2026. An Empirical Evaluation of Modern MLOps Frameworks. arXiv:2601.20415.

Naveed, H., Grundy, J., Arora, C., Khalajzadeh, H., Haggag, O., 2025. Understanding Practitioners' Perspectives on Monitoring Machine Learning Systems. arXiv:2509.25195.

Pandey, A., Sonawane, M., Mamtani, S., 2022. Deployment of ML Models using Kubeflow on Different Cloud Providers. arXiv preprint.

Punniyamoorthy, V., Agarwal, A.K., Kannan, K., Parthasarathy, A., Deshpande, A., et al., 2025. Secure and Governed API Gateway Architectures for Multi-Cluster Cloud Environments. arXiv:2512.23774.

Rubak, A., Taheri, J., 2023. Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms. In: Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing. doi:10.1145/3603166.3632165.

Shivashankar, K., Al Hajj, G.S., Martini, A., 2025. Scalability and Maintainability Challenges and Solutions in Machine Learning: Systematic Literature Review. arXiv:2504.11079.

Xu, M., Liao, J., Wu, J., He, Y., Ye, K., Xu, C., 2025. Cloud Native System for LLM Inference Serving. arXiv:2507.18007.

Zhong, Z., Xu, M., Rodriguez, M.A., Xu, C., Buyya, R., 2021. Machine Learning-based Orchestration of Containers: A Taxonomy and Future Directions. arXiv preprint.